

Projet Tutoré : Gestion de la carte Accessory Shield

I- Objectifs :

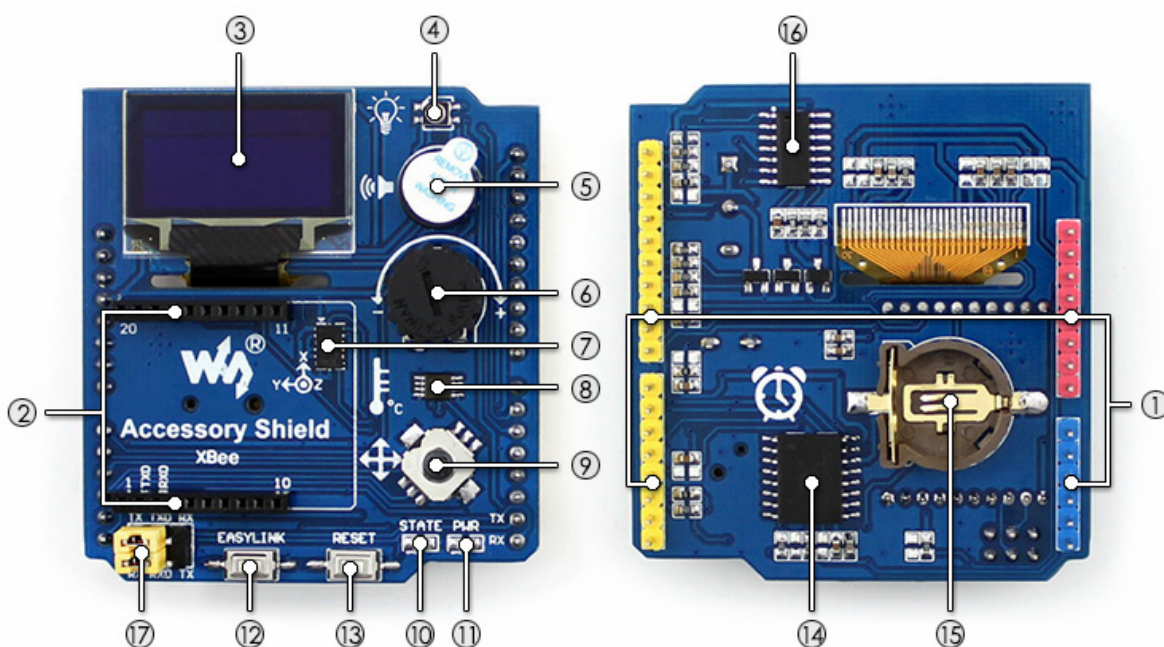
- Utilisation de plusieurs périphériques de communication série I2C/UART/SPI du STM32F446 Nucleo communiquant avec un écran LCD_I2C,
- Développement d'une application en utilisant FreeRTOS
- Gestion de la carte Accessory Shield
- Utilisation d'une communication série UART pour envoyer les données au terminal série sur PC : virtual-com UART2 via ST-Link sur STM32F446 Nucleo
- Utilisation des périphériques : I2C, UART
- Outils : teraterme.exe, carte Nucleo-64

Partie I :

Présentation de la carte Accessory Shield :

La carte Accessory Shield intègre plusieurs composants, à savoir :

- Joystick (5 directions)
- Potentiometer
- Buzzer
- RGB LED
- Capteur de Temperature
- Accelerometer 3-axes numérique
- RTC
- 0.96inch OLED
- Interface XBee



1. Arduino expansion headers : pour la connectivité avec les cartes Arduino et nucléo

2. XBee connectors : connection avec les modules XBee
3. 0.96inch OLED : 128×64 resolution
4. RGB LED : 256×256×256 colors
5. Buzzer : contrôlé en PWM
6. Potentiometer 10K
7. ADXL345 : accéléromètre 3-axes, mesurant jusqu'à $\pm 16g$
8. LM75BDP : capteur de température, résolution 0.125°C
9. Joystick : 5 directions
10. XBee module state indicator
11. Power indicator
12. XBee module EASYLINK button
13. XBee module and Arduino RESET button
14. DS3231 : RTC chip
15. CR1220 battery holder : pour RTC
16. P9813 : RGB LED driver
17. Debugging/Communication selection jumper

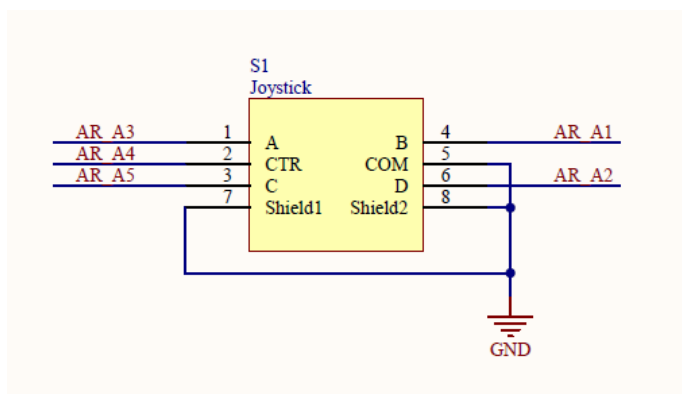
Objectif : Nous proposons dans ce projet de vous guider pour développer une application temps réelle en utilisant FreeRTOS, permettant la gestion des différents périphériques de cette carte qui communiquent via plusieurs protocoles de communication série.

Chacun d'entre eux va être gérée par une tâche. La première partie de ce projet consiste à développer les tâches séparément, ensuite nous allons définir les dépendances entre les tâches et l'intégration de la solution globale.

1- Joystick à 5 directions

Le joystick possède cinq états d'action : HAUT, BAS, GAUCHE, DROITE et ENTRÉE. Les touches correspondant à ces états sont connectées aux broches Arduino/STM32 correspondantes. L'actionnement des touches peut être évalué en vérifiant les niveaux d'E/S correspondants.

Les broches du Joystick :



A1 / PA1 : KEY_UP

A2 / PA4 : KEY_RIGHT

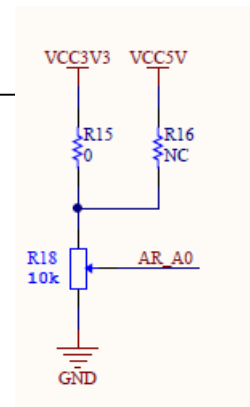
A3 / PB0 : KEY_LEFT

A4 / PC1 : KEY_ENTER

A5 / PC0:KEY_Down

Réaliser un programme en utilisant FreeRTOS permettant d'afficher sur l'écran LCD relié à l'I2C2 (Revoir la configuration et le branchement dans le TP précédent) la valeur du Joystick : quel bouton est appuyé.

Créer deux tâches, la première **Tacq_Joystick ()**, permet l'acquisition de la position du Joystick et son envoi dans une queue vers une deuxième tâche **Taffiche_LCD()** permettant son affichage sur l'écran.



2- Potentiomètre

La carte est munie d'un potentiomètre réglable de valeur 10k relié à la broche A0/PA0, configuré comme entrée de l'ADC1 : ADC1_IN0

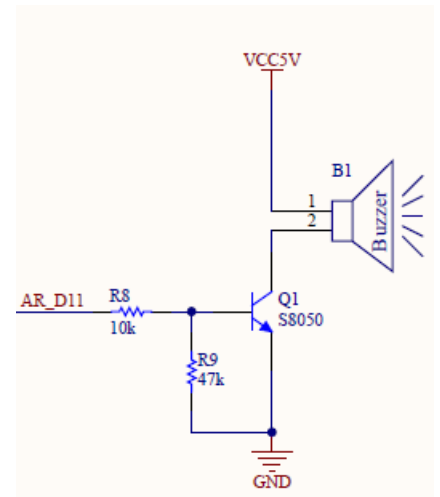
La tension délivrée par le potentiomètre est comprise entre 0 et 3.3v.

Créer la tâche **Tacq_potentiometre ()**, permettant de lire la sortie de l'ADC , la convertie en tension et l'envoi dans une queue vers la deuxième tâche **Taffiche_LCD()** permettant son affichage sur l'écran.

3- Le Buzzer

Dans les applications, un buzzer actif peut être directement connecté à l'alimentation électrique nominale pour fonctionner correctement. Cependant, un buzzer passif, tout comme un haut-parleur électromagnétique, nécessite un circuit de sortie audio pour fonctionner. Le buzzer que nous utilisons ici est un buzzer actif, capable de produire un son de fréquence simple avec une alimentation CC ou de diffuser de la musique simple avec une alimentation CA.

Dans cette application nous allons le commander en PWM, il est relié à la broche D11/PA7 qui peut être relié au TIM14_Ch1.



Application Partie I :

Réaliser un programme en utilisant FreeRTOS permettant d'afficher sur l'application teraterm (Revoir la configuration et le branchement dans le TP1_Utilisation du virtual-COM_USB_PC) :

- La valeur du Joystick : quel bouton est appuyé
- La tension délivrée par le potentiometre
- La valeur du rapport cyclique permettant la commande du Buzzer

STM32F446 Nucleo-64 board

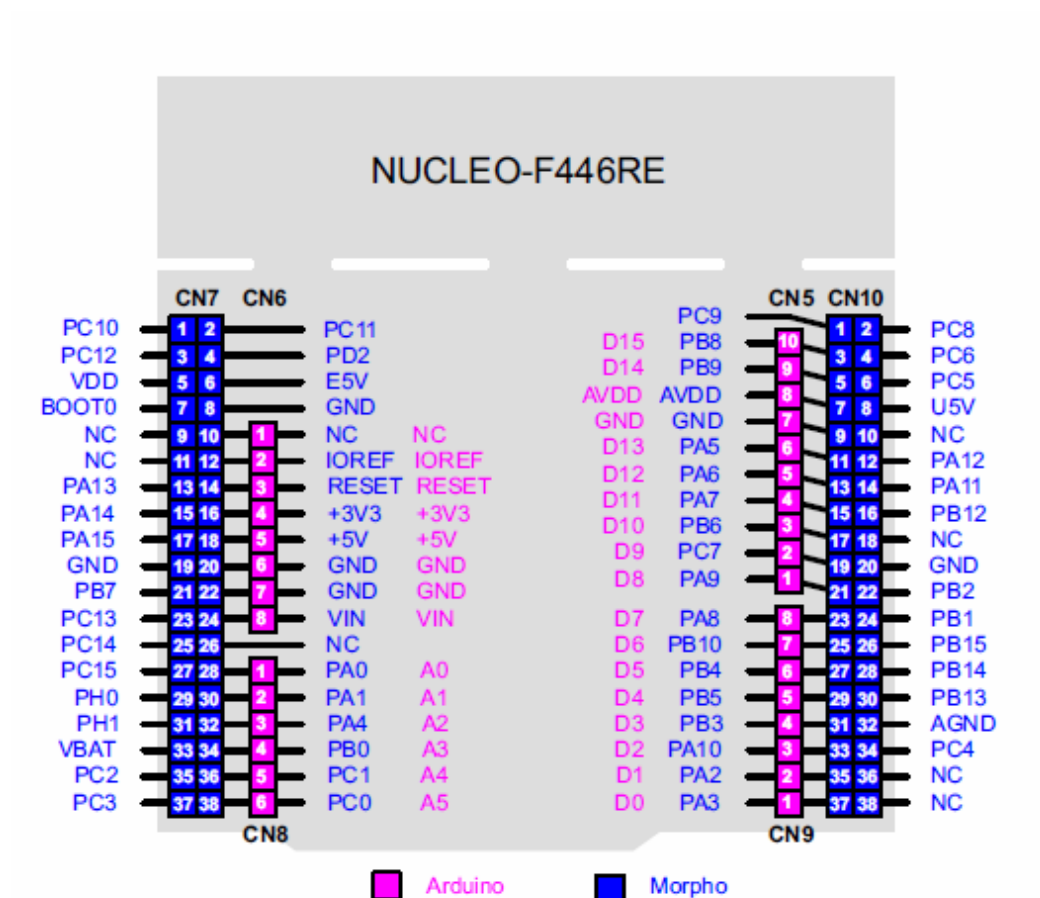


Table 19. Arduino connectors on NUCLEO-F446RE

Connector	Pin	Pin name	STM32 pin	Function
Left connectors				
CN6 power	1	NC	-	-
	2	IOREF	-	3.3V Ref
	3	RESET	NRST	RESET
	4	+3.3V	-	3.3V input/output
	5	+5V	-	5V output
	6	GND	-	ground
	7	GND	-	ground
	8	VIN	-	Power input
CN8 analog	1	A0	PA0	ADC123_IN0
	2	A1	PA1	ADC123_IN1
	3	A2	PA4	ADC12_IN4
	4	A3	PB0	ADC12_IN8
	5	A4	PC1 or PB9 ⁽¹⁾	ADC123_IN11 (PC1) or I2C1_SDA (PB9)
	6	A5	PC0 or PB8 ⁽¹⁾	ADC123_IN10 (PC0) or I2C1_SCL (PB8)
Right connectors				
CN5 digital	10	D15	PB8	I2C1_SCL
	10	D15	PB8	I2C1_SCL

Table 19. Arduino connectors on NUCLEO-F446RE (continued)

Connector	Pin	Pin name	STM32 pin	Function
CN5 digital	9	D14	PB9	I2C1_SDA
	8	AREF	-	AVDD
	7	GND	-	ground
	6	D13	PA5	SPI1_SCK
	5	D12	PA6	SPI1_MISO
	4	D11	PA7	TIM14_CH1 SPI1_MOSI
	3	D10	PB6	TIM4_CH1 SPI1_CS
	2	D9	PC7	TIM8_CH2
	1	D8	PA9	-
CN9 digital	8	D7	PA8	-
	7	D6	PB10	TIM2_CH3
	6	D5	PB4	TIM3_CH1
	5	D4	PB5	-
	4	D3	PB3	TIM2_CH2
	3	D2	PA10	-
	2	D1	PA2	USART2_TX
	1	D0	PA3	USART2_RX

1. Refer to [Table 10: Solder bridges](#) for details.